



Поисковые упражнения...



OAuth2: CSRF-защита сервера авторизации

В этом упражнении рассматривается использование CSRF-атаки на сервере авторизации OAuth2

PRO

Легко

1-2 часа

1188

Значок аутентификации / авторизации

2 видео

Лаборатория

Разверните выделенный экземпляр для этого упражнения.

↑ Развернуть

Во-первых, нужно заставить жертву перейти на вашу страницу, чтобы провести атаку с использованием oauth2:

URL -адрес:

Submit

Чтобы отметить это упражнение как выполненное, вам нужно отправить ключ, который вы получите после его выполнения:

xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx

Отправить

⚠ Пожалуйста, не публикуйте никаких общедоступных отчетов о наших ПРОФЕССИОНАЛЬНЫХ упражнениях. Спасибо!

Видео



[Введение в CSRF на сервере авторизации OAuth2](#)
03:40



[CSRF-атака на сервер авторизации OAuth2: эксплуатация](#)
07:16

Курс

Вам нравится эта бесплатная лаборатория?

Это лишь малая часть того, что вы получите с PentesterLab PRO! Получите доступ к сотням реальных лабораторий и видео, посвященных веб-хакингу, проверке кода и анализу уязвимостей CVE.

Разблокировать все лаборатории — стать PRO!

⚠ Чтобы пройти эту лабораторную работу, вам нужно подключиться к ней по протоколу HTTP, а не HTTPS. Современные браузеры автоматически переводят соединения на HTTPS, и это приводит к сбою работы. Если что-то не работает, проверьте, что ваш браузер по-прежнему использует <http://>.

ВВЕДЕНИЕ

В этом курсе подробно рассматривается использование небезопасного [OAuth2 Authorization Server](#) метода, который позволяет злоумышленнику обманом заставить жертву предоставить доступ к своей учетной записи.

[OAuth2](#) Реализовать этот метод невероятно сложно, поэтому так важно уметь находить уязвимости в его реализации.

УЯЗВИМОСТЬ

В данном случае речь идет о простой уязвимости [подделки межсайтовых запросов](#) в [Authorization Server](#). Страница, которую пользователи используют для авторизации в приложении, не проверяет [токен CSRF](#), предоставленный в рамках [POST](#) запроса.

При авторизации в приложении выполняются два шага:

1. Пользователь переходит на страницу с информацией о приложении и области действия авторизации.
2. Пользователь авторизует приложение.

Второй шаг выполняется с помощью запроса [POST](#) и должен быть защищен от CSRF-атак.

ЭКСПЛУАТАЦИЯ

Чтобы воспользоваться этой уязвимостью, вам нужно зарегистрировать учетную запись и создать [OAuth](#) приложение. Убедитесь, что вы указали [redirect URL](#) в качестве адреса своего сервера, и сохраните [client id](#) и [secret](#).

Нажав на кнопку [Authorize](#), вы получите доступ к [form](#) для авторизации приложения.



После того как вы настроите свой веб-сервер с вредоносной страницей, которая автоматически отправляет [POST](#)-запрос для авторизации приложения (с помощью [form](#)-автоподстановки), вам нужно будет заставить жертву зайти на ваш веб-сервер.

После посещения вредоносной страницы жертва должна быть перенаправлена обратно на вашу страницу (благодаря [redirect URI](#)), и вы должны получить [code](#). Вы можете использовать этот [code](#) вместе со своими [secret](#) и [client id](#) для получения [token](#). Например, если вы используете Ruby и гем [oauth2](#), следующий код позволит вам получить [token](#):

```
require 'oauth2'

callback = "[CALLBACK_URL]"
app_id = "[APPLICATION_ID]"

secret = "[SECRET]"
client = OAuth2::Client.new(app_id, secret, site: "[AUTHORIZATION_SERVER]")
client.auth_code.authorize_url(redirect_uri: callback)

code="[CODE]"
access = client.auth_code.get_token(code, redirect_uri: callback)
access.get("/api/user").parsed

puts access.token
```

Наконец, вы можете использовать [token](#) для доступа к [Resource Server](#) и получить ключ для этого задания с помощью следующей [curl](#) команды:

```
curl -H 'Authorization: Bearer [TOKEN]' [RESOURCE_SERVER]/api/keys --dump-header -
```



Если жертва авторизует ваше приложение, вам нужно как можно быстрее получить и использовать [code](#) и [token](#). Если вы будете ждать слишком долго, возможно, вам придется начать все сначала и создать новое приложение.

CONCLUSION

This exercise showed you how to exploit a vulnerability in an [OAuth2](#) implementation impacting the [Authorization Server](#). With more and more enterprises using [OAuth2](#), it's crucial to ensure that these kinds of mistakes get detected and fixed before it gets exploited.

I hope you enjoyed learning with PentesterLab.

< [PREVIOUS](#)
[SAML: Signature Wrapping II](#)

[NEXT](#)
[OAuth2: Authorization Server XSS](#)